

Team: Keyboard Gremlins

COS301 Capstone Project
University of Pretoria

Hackathon Platform



API Service Contract

Name	Student Number
Ayush Sanjith	23535424
Jasveer Ramdhaw	23537036
Rene Reddy	23558572
Heinrich Römer	23538181
Kwaku Asiedu (Sam)	18100156

Team Contact: keyboardgremlins@gmail.com

Team Creation and Management.....	3
Service Name: Create Team Service.....	3
Service Name: Request to Join Team Service.....	4
Service Name: Approve or Reject Join Request Service.....	5
Service Name: Leave Team Service.....	6
Service Name: View Team Members Service.....	7
File Storage - Specs and Submission Upload/Download.....	8
Service Name: Upload Level File Service.....	8
Service Name: Get Level File Download URL Service.....	9
Service Name: Upload Branding Asset Service.....	11
Service Name: Upload Submission Output File Service.....	12
Service Name: Upload Source Code Archive Service.....	13
Service Name: Get Submission Output Download URL Service.....	14
Service Name: Get Source Archive Download URL Service.....	15
Service Name: Get Scoring Log Download URL Service.....	16
Admin Event Management.....	17
Service Name: Create Event Service.....	17
Service Name: Get Admin Events Service.....	18
Service Name: Update Event Service.....	19
Service Name: Patch Event Status Service.....	20
Service Name: Get Event Status Service.....	21
Authorization.....	22
Service Name: Post Register User.....	22
Service Name: Post Login User.....	23

Team Creation and Management

Service Name: Create Team Service

Description: Creates a new team for a specific event and automatically adds the creator as an approved member.

Inputs:

- teamName (string) – The name of the team (must be unique within the event).
- eventId (UUID) – The identifier of the event the team belongs to.
- currentUserId (UUID) – The identifier of the authenticated user creating the team.

Outputs:

- teamId (UUID) – Unique identifier of the created team.
- teamName (string) – Name of the team.
- eventId (UUID) – Event the team belongs to.
- createdByUserId (UUID) – ID of the user who created the team.
- createdAt (datetime) – Timestamp of creation.
- status (string) – Team status (e.g., "ACTIVE").

Usage / Interaction Rules:

- Clients must send a POST request to /api/teams with a JSON body containing teamName and eventId.
- The request must include authentication (HTTP Basic Auth or JWT). The user ID is extracted from the security context.
- Preconditions:
 - Team name must not already exist for the same event.
 - The user must not already be an approved member of another team in the same event.
- On success, returns HTTP 201 Created with the team object.

- On failure, returns an appropriate HTTP error with a message.

Service Name: Request to Join Team Service

Description: Allows a participant to request joining an existing team. The request is stored as a pending membership.

Inputs:

- teamId (UUID) – Identifier of the team to join (in the URL path).
- currentUserId (UUID) – Identifier of the authenticated user requesting to join.

Outputs:

None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a POST request to /api/teams/{teamId}/join-requests.
- The request must include authentication.
- Preconditions:
 - The team must exist and be active.
 - The user must not already have a pending or approved membership for this team.
 - The team must not have reached its maximum size (currently hardcoded to 4).
- On success, returns HTTP 201 Created with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Approve or Reject Join Request Service

Description: Allows the team creator to approve or reject a pending join request from another user.

Inputs:

- teamId (UUID) – Identifier of the team (in the URL path).
- userId (UUID) – Identifier of the user whose request is being decided (in the URL path).
- approve (boolean) – true to approve, false to reject (in the request body).
- currentUserId (UUID) – Identifier of the authenticated user (must be the team creator).

Outputs:

None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a PUT request to `/api/teams/{teamId}/join-requests/{userId}` with a JSON body `{"approve": true|false}`.
- The request must include authentication.
- Preconditions:
 - The authenticated user must be the team creator.
 - The pending request must exist and have status PENDING.
 - If approving, the user must not already be an approved member of another team in the same event, and the team must still have capacity.
- On success, returns HTTP 200 OK with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Leave Team Service

Description: Allows a user to leave a team. If the user is an approved member, the membership status changes to "LEFT". If the user has only a pending request, the membership record is deleted. If the last approved member leaves, the team becomes inactive.

Inputs:

- teamId (UUID) – Identifier of the team to leave (in the URL path).
- currentUserId (UUID) – Identifier of the authenticated user leaving.

Outputs:

None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/teams/{teamId}/members.
- The request must include authentication.
- Preconditions:
 - The user must be a member of the team (either pending or approved).
- On success, returns HTTP 204 No Content with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: View Team Members Service

Description: Retrieves the list of approved members of a team, including their names, emails, join dates, and team role (LEADER or MEMBER).

Inputs:

- teamId (UUID) – Identifier of the team (in the URL path).

Outputs:

- A list of member objects, each containing:
 - userId (UUID) – User identifier.
 - fullName (string) – Concatenated first and last name.
 - email (string) – User's email address.
 - joinedAt (datetime) – When the user joined.
 - role (string) – Either "LEADER" or "MEMBER".

Usage / Interaction Rules:

- Clients must send a GET request to /api/teams/{teamId}/members.
- Authentication may be required (depending on security configuration).
- Preconditions:
 - The team must exist.
- On success, returns HTTP 200 OK with the JSON array of members.
- On failure (e.g., team not found), returns an appropriate HTTP error with a message.

File Storage - Specs and Submission Upload/Download

Service Name: Upload Level File Service

Description: Allows an Admin to upload an input or resource file for a specific event level. The file is stored in Azure Blob Storage and the storage key is returned for database persistence.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- levelId (string) - Identifier of the level (in the URL path).
- file (multipart/form-data) - The file to upload.

Outputs:

- storageKey (string) - The blob path persisted to levelfiles.storage_key in the database.
- blobUrl (string) - The raw Azure blob URL.

Usage / Interaction Rules:

- Clients must send a POST request to `/api/storage/events/{eventId}/levels/{levelId}/files` with a multipart/form-data body containing a field named file.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with storageKey and blobUrl.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Level File Download URL Service

Description: Returns a time-limited presigned SAS URL for a participant or admin to download a level input file directly from Azure Blob Storage.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- levelId (string) - Identifier of the level (in the URL path).
- filename (string) - The filename to retrieve (in the URL path).

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes.

Usage / Interaction Rules:

- Clients must send a GET request to
/api/storage/events/{eventId}/levels/{levelId}/files/{filename}.
- The request must include authentication (JWT). Admin or Participant role required.
- The presigned URL allows direct download from Azure without exposing credentials.
- On success, returns HTTP 200 OK with the url field.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Upload Solver File Service

Description: Allows an Admin to upload a solver file for a specific event. Supports versioning so that hotfix solver versions can be uploaded without overwriting previous versions.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- version (integer) - The solver version number (query parameter).
- file (multipart/form-data) - The solver file to upload.

Outputs:

- storageKey (string) - The blob path persisted to solversion.storage_key in the database.
- blobUrl (string) - The raw Azure blob URL.
- version (string) - The version number that was uploaded.

Usage / Interaction Rules:

- Clients must send a POST request to `/api/storage/events/{eventId}/solver?version={versionNumber}` with a multipart/form-data body containing a field named file.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with storageKey, blobUrl, and version.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Upload Branding Asset Service

Description: Allows an Admin to upload a branding asset such as a logo or banner image for a specific event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- file (multipart/form-data) - The image file to upload.

Outputs:

- storageKey (string) - The blob path of the uploaded asset.
- blobUrl (string) - The raw Azure blob URL.

Usage / Interaction Rules:

- Clients must send a POST request to `/api/storage/events/{eventId}/branding` with a multipart/form-data body containing a field named file.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with storageKey and blobUrl.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Upload Submission Output File Service

Description: Allows a Participant to upload their solution output file for a specific submission. This is the artifact that the solver grades. The storage key is persisted to `submissions.output_storage_key` in the database.

Inputs:

- `eventId` (UUID) - Identifier of the event (in the URL path).
- `teamId` (UUID) - Identifier of the team (in the URL path).
- `submissionId` (string) - Identifier of the submission (in the URL path).
- `file` (multipart/form-data) - The solution output file.

Outputs:

- `storageKey` (string) - The blob path persisted to `submissions.output_storage_key` in the database.
- `blobUrl` (string) - The raw Azure blob URL.

Usage / Interaction Rules:

- Clients must send a POST request to `/api/storage/events/{eventId}/teams/{teamId}/submissions/{submissionId}/output` with a multipart/form-data body containing a field named `file`.
- The request must include authentication (JWT). Participant role required.
- The platform never executes participant code. Only this output file is passed to the solver.
- On success, returns HTTP 200 OK with `storageKey` and `blobUrl`.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Upload Source Code Archive Service

Description: Allows a Participant to upload a zipped source code archive alongside a submission. The archive is stored for post-event verification and auditing to confirm the submitted output is reproducible. The storage key is persisted to `submissions.source_code_storage_key` in the database.

Inputs:

- `eventId` (UUID) - Identifier of the event (in the URL path).
- `teamId` (UUID) - Identifier of the team (in the URL path).
- `submissionId` (string) - Identifier of the submission (in the URL path).
- `file` (multipart/form-data) - A .zip archive of the source code.

Outputs:

- `storageKey` (string) - The blob path persisted to `submissions.source_code_storage_key` in the database.
- `blobUrl` (string) - The raw Azure blob URL.

Usage / Interaction Rules:

- Clients must send a POST request to `/api/storage/events/{eventId}/teams/{teamId}/submissions/{submissionId}/source` with a multipart/form-data body containing a field named `file`.
- The request must include authentication (JWT). Participant role required.
- The source archive is never executed by the platform. It is stored for auditing purposes only.
- On success, returns HTTP 200 OK with `storageKey` and `blobUrl`.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Submission Output Download URL Service

Description: Returns a presigned SAS URL for downloading a submission output file from Azure Blob Storage.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- teamId (UUID) - Identifier of the team (in the URL path).
- submissionId (string) - Identifier of the submission (in the URL path).
- filename (string) - The output filename (in the URL path).

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes.

Usage / Interaction Rules:

- Clients must send a GET request to
/api/storage/events/{eventId}/teams/{teamId}/submissions/{submissionId}/output/{filename}.
- The request must include authentication (JWT). Admin or Participant role required.
- On success, returns HTTP 200 OK with the url field.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Source Archive Download URL Service

Description: Returns a presigned SAS URL for downloading a source code archive from Azure Blob Storage. Admin only - used for post-event auditing to verify that the submitted output is reproducible by the team's code.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- teamId (UUID) - Identifier of the team (in the URL path).
- submissionId (string) - Identifier of the submission (in the URL path).
- filename (string) - The archive filename (in the URL path).

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes.

Usage / Interaction Rules:

- Clients must send a GET request to
/api/storage/events/{eventId}/teams/{teamId}/submissions/{submissionId}/source/{filename}.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with the url field.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Scoring Log Download URL Service

Description: Returns a presigned SAS URL for downloading a scoring log file from Azure Blob Storage.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- submissionId (string) - Identifier of the submission (in the URL path).
- filename (string) - The log filename (in the URL path).

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes.

Usage / Interaction Rules:

- Clients must send a GET request to
/api/storage/events/{eventId}/submissions/{submissionId}/logs/{filename}.
- The request must include authentication (JWT). Admin or Participant role required.
- On success, returns HTTP 200 OK with the url field.
- On failure, returns an appropriate HTTP error with a message.

Admin Event Management

Service Name: Create Event Service

Description: Creates a new hackathon event with specific constraints

Inputs:

- EventRequest object payload in the request body that contains:
 - Name (string) - The title of the event.
 - registrationKey (string) - The optional passkey for private events.
 - teamSizeLimit (short) - The maximum number of participants per team.
 - startDateTime (OffsetDateTime) - The starting time and date of the event.
 - Duration (integer) - the total duration of an event in hours.
 - Description (string) - Information about the specific hackathon event.
 - Visibility (string) - The visibility of the hackathon event to the participants.
 - Status (string) - The lifecycle of an event, it can be ACTIVE or INACTIVE as an example.

Output:

- eventId (UUID) - Automatically generated unique database identifier for the event.
- createdByUserId (UUID) - The identifier of the admin that created the event.
- Name (string) - The configured title of the event.
- registrationKey (string) - The password for the event or null if Status is not private.
- teamSizeLimit (short) - The configured team size constraint for the event.
- startDateTime (OffsetDateTime) - The configured start date for the event.
- Duration (int) - The configured duration of the event.
- Description (string) - The configured description of the event.
- Visibility (string) - The configured visibility of the event.
- Status (string) - The configured status of the event.

Usage / Interaction Rules:

- Clients must send a POST request to /api/admin/events.
- The body of the request must include a valid JSON object matching the properties expected by EventRequest.
- The client must hold a valid admin role (ROLE_ADMIN).
- On success: Returns HTTP 201 Created with the newly created event in the response body.
- On failure: Returns appropriate HTTP error payload.

Service Name: Get Admin Events Service

Description: Retrieves a list of all hackathon events that were created by a specific admin, this is done by using their specific UUID and matching it to the createdByUserId field.

Inputs:

- createdByUserId (UUID) - the unique identifier of the admin whose events are being retrieved.

Output:

- Returns a List<Event> array in the response body. Each event in the body is a valid event that satisfies the rules in the Create Event Service. If the admin has not yet created any events an empty array will be returned.

Usage / Interaction Rules:

- Client must send a GET request to the /api/admin/events.
- The client must hold a valid admin role (ROLE_ADMIN).
- On success: Returns HTTP 200 OK containing the JSON array of the matching event entities (or an empty array if no events could be found for an admin).

- On failure: Returns HTTP 403 Forbidden if the client does not have a `ROLE_ADMIN`.
- On failure: HTTP 400 if the id provided is malformed or not a valid UUID.

Service Name: Update Event Service

Description: Performs a complete overwrite of an existing hackathon event's configured data using the provided event identifier and request details.

Input:

- EventRequest object payload in the request body that contains:
 - Name (string) - The title of the event.
 - registrationKey (string) - The optional passkey for private events.
 - teamSizeLimit (short) - The maximum number of participants per team.
 - startDateTime (OffsetDateTime) - The starting time and date of the event.
 - Duration (integer) - the total duration of an event in hours.
 - Description (string) - Information about the specific hackathon event.
 - Visibility (string) - The visibility of the hackathon event to the participants.
 - Status (string) - The lifecycle of an event, it can be `ACTIVE` or `INACTIVE` as an example.

Output:

- Returns the updated and fully persistent Event entity object with the updated information matching that specified eventId.

Usage / Interaction Rules:

- Clients must send a PUT request to `/api/admin/events/{id}` where id is the UUID of the specific event that exists on the database.
- The event must already be present in the database.
- On success: Return HTTP 200 OK containing the updated Event entity representation inside the response body payload.

- On failure: On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.
- On failure: HTTP 404 Not Found if the passed in eventId is not in the database or could not be found.

Service Name: Patch Event Status Service

Description: Performs a partial update on an existing hackathon event to modify its status, visibility or registrationKey if needed. The rest of the event stays unchanged.

Input:

- An EventRequest object payload in the request body that contains:
 - Visibility (string) - The new visibility state.
 - Status (string) - the new status state.
 - registrationKey (string) - the updated passkey to the event (this is optional and only required if the event visibility is private)

Output:

- Returns the modified and fully persistent Event entity object with the updated information matching that specified eventId.

Usage / Interaction Rules:

- Client must send a PATCH request to /api/admin/events/{id}/status where id represents the target events id.
- If the visibility is private then a registrationKey must be provided, otherwise it will automatically be set to null.
- On success: Return HTTP 200 OK containing the modified Event entity representation inside the response body payload.
- On failure: Returns 404 Not Found if the eventId does not exist in the database.
- On failure: On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.

Service Name: Get Event Status Service

Description: Retrieves a lightweight summary containing only the current operational status and event visibility of a specific hackathon event.

Input:

- eventId (UUID) - the unique identifier of the specific event we are inspecting.

Output:

- Status (string) - the current status of the event we are inspecting.
- Visibility (String) - the current visibility state of the event we are inspecting.

Usage / Interaction Rules:

- Clients must send a GET request to `/api/admin/events/{id}/status` where id is the UUID of the specified event.
- The endpoint is restricted to authenticated users with the role admin (ROLE_ADMIN).
- On success: HTTP 200 OK containing the EventStatusResponse JSON payload
- On failure: Returns 404 Not Found if the eventId does not exist in the database.
- On failure: On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.

Authorization

Service Name: Post Register User

Description: Registers a user by entering their credentials into the system. Assigns a Participant role by default.

Input:

- firstName (string) - First name of the user.
- lastName (String) - Last name of the user.
- Email (String) - Email of the user.
- Password (String) - Password of the user.

Output:

- Token (string) - JWT token.
- UserId (UUID) - ID of the user.
- firstName (string) - First name of the user.
- lastName (String) - Last name of the user.
- Email (String) - Email of the user.
- Role (string) - The user's role.

Usage / Interaction Rules:

- Clients must send a POST request to /api/auth/register.
- The endpoint assigns a JWT token.
- On success: HTTP 200 OK containing the AuthResponse JSON payload
- On failure: Returns 409 Conflict if the user is already registered.

Service Name: Post Login User

Description: Login using saved email and password.

Input:

- Email (string) - Users email.
- Password (string) - Users password.

Output:

- Token (string) - JWT token.
- UserId (UUID) - ID of the user.
- firstName (string) - First name of the user.
- lastName (String) - Last name of the user.
- Email (String) - Email of the user.
- Role (string) - The user's role.

Usage / Interaction Rules:

- Clients must send a POST request to /api/auth/login.
- The endpoint assigns a JWT token.
- On success: HTTP 200 OK containing the AuthResponse JSON payload
- On failure: Returns 404 Not Found if the user is not registered.